

Process Improvement Framework (PDCA)

Doc ID: HH-CMM-02 **Version:** 1.0 **Effective Date:** 2026-08-29 **Owner:** Venkat Narasimha (Processbricks) **Review Cycle:** Annual (next: 2027-08-29) **Last Reviewed:** 2026-08-29

Classification: Internal **One-line summary:** We commit to running at least one Plan-Do-Check-Act cycle per quarter, documenting it, and using the documented cycle as the primary evidence for CMM Level 5.

1. Purpose

Maturity Level 5 is not “we never have problems” — that’s impossible. Level 5 is “we change the process because the data says to, and we can prove we changed it”. PDCA (Plan-Do-Check-Act) is the public, well-understood way to do that.

Without a PDCA framework, improvements happen by accident and are lost when the person who made them leaves. With a framework, improvements are:

- **Visible** — written down at docs/phase6/06-process-improvements/{N}-{slug}.md.
- **Testable** — the hypothesis and the metric are explicit, so “did this work?” has an answer.
- **Cumulative** — a future PDCA can build on a prior PDCA’s result instead of rediscovering it.
- **Aligned with industry vocabulary** — auditors, future hires, and partners who know PDCA can read our cycles in five minutes.

This is the practice that moves us from “we got lucky this time” (Level 3) to “we improve systematically” (Level 5). It complements 12-defect-prevention (which is about catching defects) and 11-quantitative-management (which provides the data PDCA needs).

2. Scope

2.1 In scope

- **What PDCA is** — origin, the 4 steps, why it works.
- **PDCA applied to Haritha** — the Haritha-specific interpretation: cadence, owners, log location.
- **Worked examples** — 3+ historical PDCA cycles reconstructed from LEARNINGS.md.
- **Future PDCA opportunities** — 3-5 specific candidates identified for 2026-Q4.
- **Templates and tooling** — markdown template, git workflow, log directory.
- **Cadence and KPI** — minimum 1 cycle per quarter; tracked in 09-cmm-maturity §5.

2.2 Out of scope

- **Metrics definitions** — see 11-quantitative-management.
- **RCA techniques for finding the problem** — see 12-defect-prevention. PDCA assumes you already know the problem; RCA finds it.
- **Change management approval flow** — see 05-process/05.1-change-management. A PDCA cycle that changes production must still go through change management unless it’s an emergency hotfix.
- **Incident-specific PDCA** — when an incident triggers a PDCA cycle, the post-mortem in 05.2 is the canonical record; the PDCA cycle captures the *improvement*, not the incident itself.

3. Policy Statement

3.1 What we commit to

Haritha Hospitals commits to:

1. **Minimum one PDCA cycle per quarter.** Q1/Q2/Q3/Q4 each produce at least one completed cycle (or one in-progress cycle with documented Plan step at quarter-end).
2. **Every cycle logged** at docs/phase6/06-process-improvements/{NN}-{slug}.md, numbered sequentially starting at 01-.
3. **Every cycle has all four steps documented** — Plan, Do, Check, Act. A cycle that doesn't reach Act (e.g., the hypothesis failed) is still logged, with the "Act" step being "revert + new hypothesis".
4. **Cycles cite lessons** — when a cycle ends in a lesson learned, that lesson is added to LEARNINGS.md with the cycle ID as a reference.
5. **Cycles cite runbooks** — when a cycle changes a runbook, the runbook footer is updated with the cycle ID.
6. **Annual summary** — once a year, Venkat reviews the cycle log and writes a 1-page summary at docs/phase6/06-process-improvements/YEAR-summary.md. The summary feeds the 09-cmm-maturity-assessment §3a re-scoring.

3.2 The four steps (canonical)

PDCA was popularised by W. Edwards Deming in the 1950s (building on Shewhart's 1930s "Plan-Do-See"). It's a 4-step iterative method for improving a process. The canonical interpretation:

Step	What it is	Haritha-specific deliverable
Plan	Identify the problem or goal. Hypothesise a change. Define the metric that will tell you if the change worked.	A 1-paragraph problem statement + a hypothesised change + a measurable success criterion.
Do	Implement the change on a small scale. Document the implementation carefully (what was changed, when, by whom, with what backup).	A diff or commit, a timestamp, a verifier (parent or subagent), and a rollback plan.
Check	Measure the result. Compare to the hypothesis. Did it work? Why or why not?	The metric value(s) before and after; a pass/fail verdict; an explanation of any gap.
Act	If it worked: standardise, document, expand. If it didn't: revise the hypothesis and start a new cycle.	Either (a) a "standardise" commit + runbook update + LEARNINGS entry, or (b) a "new hypothesis" + a fresh cycle.

Why it works: the loop closes the gap between "we made a change" and "we know whether the change helped". Without the Check step, improvements are wishes. Without the Act step, learnings don't propagate.

3.3 PDCA vs ad-hoc improvements — the difference

Dimension	Ad-hoc improvement	PDCA cycle
Problem statement	"Something was broken"	"Problem X exists because of Y; success means Z is measurable"
Hypothesis	Implicit	Explicit, testable
Metric	None	Defined up-front
Result	"I think it worked"	"Metric went from A to B; we are 95% confident"
Reusability	Depends on the person	Documented; the next cycle can build on it
Audit trail	Maybe a Slack message	A markdown file + commits + LEARNINGS entry

3a. Current State (as of 2026-08-29)

3a.1 What we have TODAY

PDCA component	Component	Where it lives	Status
Improvement loop	Backups hardened over multiple iterations	LEARNINGS #79, #80, #113, #114	Done (informal PDCA)
Improvement loop	Gunicorn restart pattern after install-app	LEARNINGS #46, #153	Done (informal PDCA)
Improvement loop	Idempotent master-data migration	LEARNINGS #157	Done (informal PDCA)
Improvement loop	Heartbeat freshness rule (probe vs carry-forward)	LEARNINGS #90	Done (informal PDCA)
Cycle log directory	docs/phase6/06-process-improvements/	not yet created	Not Started
PDCA template	Markdown template	not yet written	Not Started
Quarterly cadence	Owner-driven, not tracked	this doc §3.1	Not Started
Annual summary	Year-end review	not yet written	Not Started

3a.2 What is WORKING

- **The improvement loop itself works.** Multiple LEARNINGS entries are clear evidence of Plan→Do→Check→Act having happened. The gap is not that we don't improve; it's that we don't *track* improvement as PDCA.
- **Post-mortems feed LEARNINGS.md.** Every SEV-1/2 post-mortem writes at least one lesson. PDCA cycles can pull from this feed.
- **Subagents can run cycles.** A PDCA cycle's "Do" step often involves a subagent making a change; the parent-verify pattern (LEARNINGS #72) is the cycle's "Check" step.

3a.3 Known GAPS

1. **No formal cycle log.** Improvements live in LEARNINGS.md and runbooks but not as discrete cycles. Creating the directory + template is the first concrete action.

2. **No quarterly tracking.** “Did we run a cycle this quarter?” is not answered anywhere. Quarterly cadence enforcement will live in the annual summary review.
3. **No cycle-to-runbook feedback automation.** A cycle that changes a runbook must be manually cross-linked. Future improvement: a script that diffs the cycle log against runbook footers.
4. **No “failed cycle” log.** Cycles whose hypothesis failed are rarely written up. We learn from successes; we should also learn from failures. PDCA template explicitly captures both.
5. **No cycle ownership.** Each cycle should have a named owner (the person who commits to closing it). Without ownership, cycles drift.
6. **No metric linkage.** A cycle’s Check step needs a metric; metrics live in 11-quantitative-management. The two docs must cross-reference.

These gaps are explicit v1 scope. Listing them is transparency, not apology.

3b. Concrete Examples (Haritha history)

Three historical PDCA cycles reconstructed from LEARNINGS.md. They happened; they just weren’t called PDCA. The reconstructions below show what each cycle *would* look like as a formal PDCA record.

Example 1 — Backup hardening PDCA (reconstructed from LEARNINGS #79, #80, #113, #114)

Cycle A — original silent-failure fix

- **Plan.** Problem: `prod_backup.sh` exits 0 with no useful log line on failure (LEARNINGS #113). Goal: every successful run produces a `BACKUP_OK` sentinel; every failed run exits non-zero with a clear error. Hypothesis: replace `$(ls *.tar.gz)` with deterministic path + add sentinel. Metric: presence of `BACKUP_OK` line at end of log; presence of `*.tar.gz` at offsite.
- **Do.** 2026-08-19 ~19:25 IST. Edited `prod_backup.sh` to bundle 4 files → 1 tarball, then `rsync` that one file, then `echo BACKUP_OK`. Restarted cron slot manually to test.
- **Check.** 2026-08-19 21:00 IST. Last log line: `BACKUP_OK sha=... remote=...`. Offsite `rsync` contains the tarball. □
- **Act.** Standardised. Updated 04.3-disaster-recovery §“Backup verification” to read the sentinel. Added LEARNINGS #113.

Cycle B — daily log-tail probe

- **Plan.** Problem: even after the fix, silent failures could hide for days if cron reports “completed normally” (LEARNINGS #114). Goal: a daily probe that tails the actual log file and alerts if last `BACKUP_OK` is > 26h old. Hypothesis: 09:00 IST daily `tail -n 5` is enough. Metric: time-to-detect on a future regression.
- **Do.** 2026-08-20 09:30 IST. Added 09:00 IST cron: `tail -n 5 /var/log/pberpprod_backup.log` and check for `BACKUP_OK`. Wired into heartbeat.
- **Check.** 2026-08-27: probe ran every day; detected the 2026-08-10..18 streak only when Venkat reviewed the heartbeat manually — i.e., probe ran but didn’t alert loudly enough. △ Partial pass.
- **Act.** Cycle C below.

Cycle C — louder alert

- **Plan.** Problem: probe ran but Venkat didn’t see the alert path. Hypothesis: add Telegram ping on `BACKUP_OK` age > 26h. Metric: time-to-Venkat-notification on a regression.

- **Do.** 2026-08-27. Added Telegram alert path.
- **Check.** Pending (no regression since).
- **Act.** Standardise pending first regression.

What this 3-cycle sequence teaches: a single PDCA cycle rarely solves a problem fully. The first fix is rarely the right one. Cycles 1→2→3 each refined the previous. PDCA isn't one-and-done; it's a loop that tightens over time.

Example 2 — Unicorn --preload restart PDCA (LEARNINGS #46, #153)

- **Plan.** Problem: after bench install-app, unicorn's --preload sys.path is frozen; new app's .pth file is invisible (LEARNINGS #46 observed pre-2026-08; LEARNINGS #153 confirmed on 2026-08-29). Goal: every install-app includes a backend container restart. Hypothesis: add a step to the deploy runbook that runs `docker restart erp-{env}-backend-1` after every bench install-app. Metric: zero ModuleNotFoundError: No module named 'X' 500s after install-app.
- **Do.** 2026-08-29 04:30 IST (post-outage). Updated 04.1-deployment's "Post-install restart" with explicit `docker restart` step.
- **Check.** 2026-08-29 06:00 IST. Re-installed haritha_hospital on both envs (already idempotent, but tested); both came up with 200s and login working. ☐
- **Act.** Standardised. Added the rule to MEMORY.md "Critical Lessons" section. Runbook updated. LEARNINGS #153 written.

What this teaches: a PDCA cycle that came out of an incident is the most valuable kind. The 2026-08-29 outage cost ~10 min of downtime; the cycle that emerged prevents it from happening again.

Example 3 — Idempotent master-data migration PDCA (LEARNINGS #157)

- **Plan.** Problem: master-data migration script from pberpprod → pberpqa needed to be safe to re-run (idempotent). First version inserted-only; second run created duplicates. Hypothesis: use `frappe.get_doc().save()` upsert pattern (check `frappe.db.exists()` first). Metric: re-runs produce zero duplicates.
- **Do.** 2026-08-29. Rewrote `scripts/migrate_master_data.py` per the upsert pattern.
- **Check.** Ran script 3 times consecutively; each run produced identical DB state (verified by row counts); no duplicate-key errors. ☐
- **Act.** Standardised. LEARNINGS #157 documents the pattern with 10 gotchas. Script lives at the canonical path.

What this teaches: idempotency is a PDCA outcome, not a starting assumption. The first attempt (insert-only) failed the re-run test; the second attempt (upsert) passed.

Example 4 — Heartbeat freshness PDCA (LEARNINGS #90)

- **Plan.** Problem: heartbeat reported stale data carried forward from a prior day. Disk was reported as 77% when actual was 95% (a near-miss). Hypothesis: enforce fresh probe every ≤ 24 h for drift-prone metrics. Metric: max carry-forward interval for disk, memory, container count.
- **Do.** 2026-08-21. Added rule "fresh probe every ≤ 24 h for drift-prone metrics" to 04.2-daily-ops and heartbeat subagent brief.
- **Check.** 2026-08-22..29: heartbeat probes ran fresh every day; no carry-forward regression. ☐
- **Act.** Standardised. Rule is now permanent; LEARNINGS #90.

Example 5 — Anti-example: the CapitalCase color fix (2026-08-28, three attempts)

This is a counter-example: a fix that was *not* PDCA, with measurable cost. - **Cycle 1 (wrong)**. “Submit draft SAs” — Venkat dismissed subagent’s correct hypothesis. Fix not deployed. - **Cycle 2 (half)**. “Hex codes → Tailwind CapitalCase” — looked like progress; JS still crashed. - **Cycle 3 (right)**. “CapitalCase → lowercase” — actual fix.

What this teaches: dismissing a subagent’s correct diagnosis (LEARNINGS #72 violation in spirit) costs cycles. Three attempts; one was right; two were not. A PDCA log would have captured the hypothesis + Check step on each attempt, making the pattern visible.

4. Responsibilities

Role	Responsibilities
Venkat Narasimha (Owner)	Approves new PDCA cycles. Reviews annual summary. Decides which gaps are PDCA-priority vs back-burner. Owns the assessment loop (cross-link 09-cmm-maturity §4).
Processbricks admin	Runs PDCA cycles assigned to them. Logs cycles in the cycle directory. Verifies Check steps (parent-verify). Updates runbooks + LEARNINGS.md when Act step standardises.
Subagents (automation)	Can execute the Do step of a cycle (with parent approval). Can run the Check step’s probes. Cannot declare a cycle complete — only the named owner can.
Future operators	Read the cycle log first when joining. A new operator following the cycles should be able to reach the same standard of process knowledge as the operator who wrote them.

5. Compliance Measurement

Check	Frequency	Owner	Source of truth
Minimum 1 cycle logged per quarter	Quarterly	PA	cycle directory listing
Every cycle has all 4 steps documented	Per cycle	PA	cycle file schema
Every cycle cites lessons + runbooks	Per cycle	PA	grep cycle directory for cross-refs
Failed cycles are also logged	Per cycle	PA	cycle directory content
Annual summary written	Annually	VN	docs/phase6/06-process-improvements/YEAR-summary.md
Cycle-to-runbook update lag	Per cycle	PA	git log timestamp diff

Check	Frequency	Owner	Source of truth
Maturity assessment updated with cycle outcomes	Annually	VN	09 §3a.2 re-scoring

KPI dashboard:

KPI	Target	Source
Cycles completed per quarter	≥ 1	cycle directory
Cycle completion rate (reached Act vs abandoned at Do)	≥ 80%	cycle directory
Average cycle duration (Plan → Act)	≤ 30 days	cycle file mtimes
Lessons-per-cycle (output)	≥ 1 (when the cycle standardises something)	LEARNINGS.md entries citing cycle
Runbook updates from cycles	≥ 1 per cycle that standardises	runbook git log

6. Exceptions

1. **Emergency hotfixes** (04.1-deployment §“Emergency hotfix”) are exempt from the “logged as PDCA cycle” requirement. The post-mortem captures the lesson; the PDCA cycle, if any, is created within 7 days post-incident.
2. **Cycles < 1 day** can be summarised inline in the post-mortem or in a chat log instead of a dedicated file, at owner’s discretion. (e.g., “tightened a script’s regex” — log inline; “changed the entire backup architecture” — log as full PDCA.)
3. **Cycles that are pure research** (no Do step, e.g., “investigate whether X is feasible”) are out of PDCA scope; use a spike or RFC instead.
4. **All other exceptions** follow 01-info-security §6.

6a. Edge Cases & Decision Matrix

Edge case 1 — A cycle’s hypothesis fails

- **Trigger.** Cycle 1 ran; Check step shows hypothesis was wrong.
- **Decision matrix.** Document the failure in the Act step. Either:
 - Start Cycle 2 with a revised hypothesis (this is normal — multiple cycles is the rule).
 - Revert the Do step and close the cycle (if the change is harmful).
- **Default action.** Log the failure. The annual summary explicitly counts failed cycles — a year with 0 failed cycles is suspect (means we only attempt safe changes).

Edge case 2 — A cycle takes longer than a quarter

- **Trigger.** Plan step in Q1; Do step spans Q1→Q2; Check in Q3.
- **Decision matrix.** This is fine. The cycle is logged when started; “in progress across quarters” is not a violation. The quarterly count counts *started* or *completed*, not both; pick one and document.

Counting strategy	When to use
Count completed cycles	When we want to enforce “every quarter produces a finished improvement”
Count started cycles	When we want to track activity, not just finish-line
Default	Count started. Document explicitly when starting a cycle that may span quarters.

Edge case 3 — Two cycles want to change the same thing

- **Trigger.** Admin starts “improve backup script”; Venkat starts “improve backup script” on the same day.
- **Decision matrix.** Stop both. Reconvene. Either merge the two into one cycle (one owner, combined Plan step) or sequence them (cycle 1 produces a baseline; cycle 2 builds on it).
- **Default action.** Merge unless the owners disagree on the hypothesis. Document the merge in the surviving cycle.

Edge case 4 — A cycle is started but the owner leaves

- **Trigger.** Admin started a cycle; admin transitioned out; nobody owns it.
- **Decision matrix.** Reassign owner within 30 days OR close the cycle with Act = “abandoned (owner transitioned)”. Either way, the cycle is not in limbo.
- **Default action.** Venkat reassigns or closes. The cycle log is the source of truth for “is this in flight?”.

Edge case 5 — A cycle’s metric can’t be measured

- **Trigger.** Plan step says “metric = deployment frequency”. After 60 days we have no easy way to count deployments.
- **Decision matrix.** Two paths:
 - Define the metric properly (likely via 11-quantitative-management — add the metric).
 - If the metric can’t be defined, the cycle’s Plan step is incomplete. Close it; document the lesson as “metric not measurable”; new cycle will have a measurable plan.
- **Default action.** Define the metric or close. Don’t run cycles whose success can’t be measured.

Edge case 6 — A cycle introduces a regression

- **Trigger.** Cycle’s Do step improves X but breaks Y.
- **Decision matrix.** The cycle is not a failure; it’s a finding. Act step = “revert Y; keep X if X’s improvement is bigger than Y’s regression; or close cycle entirely if X is small”. Document the trade-off.
- **Default action.** Use the post-mortem format (05.2) to capture the regression. The cycle log + post-mortem together tell the full story.

Edge case 7 — A cycle’s success can’t be replicated

- **Trigger.** Cycle says “metric improved”; 30 days later, the metric regresses.
- **Decision matrix.** PDCA’s Check step captures the snapshot, not the trend. A regression doesn’t invalidate the cycle’s success — it starts a new cycle asking “why did the metric regress?”. Document both cycles.

- **Default action.** Don't claim "permanent fix" in the Act step unless there's a control limit enforcing it. "Fixed for 30 days" is a snapshot; "fixed permanently" requires ongoing measurement.

Edge case 8 — A cycle produces no LEARNINGS entry

- **Trigger.** Act step standardises something that's already documented; no new lesson.
- **Decision matrix.** Fine. Not every cycle produces a lesson. A cycle that hardens an existing pattern (e.g., "added retry to backup script that was already retry-tolerant") might just be a runbook update.
- **Default action.** Skip LEARNINGS entry; cite the runbook update in the cycle's Act step.

7. Related Documents

- 09-cmm-maturity-assessment.md — HH-CMM-01 — The assessment that PDCA cycles feed into.
- 11-quantitative-management.md — HH-CMM-03 — Metrics; the Check step depends on metric definitions.
- 12-defect-prevention.md — HH-CMM-04 — RCA techniques; finding the problem before PDCA starts.
- 05-process/05.1-change-management.md — Approval flow for cycles that change production.
- 05-process/05.2-post-mortem.md — Post-mortem template; cycles triggered by incidents start here.
- 04-runbooks/ — Where standardised cycles land their changes.
- ../../../../learnings/LEARNINGS.md — Cycles cite lessons; lessons cite cycles.
- ../../../../MEMORY.md — Tech stack + standing rules.

8. Revision History

Version	Date	Author	Changes
1.0	2026-08-29	venkat-narasimha	Initial

9. Implementation Checklist

Concrete actions derived from this policy. Owner initials: VN = Venkat Narasimha; PA = Processbricks admin. Status as of 2026-08-29.

Immediate (this week)

- ☐ **Create docs/phase6/06-process-improvements/ directory** + add to git. Owner: PA. Target: 2026-09-05. Status: Not Started.
- ☐ **Author the PDCA cycle template** at docs/phase6/06-process-improvements/00-template.md. Owner: VN. Target: 2026-09-05. Status: Not Started.
- ☐ **Backfill three historical cycles** as 01-backup-hardening.md, 02-gunicorn-restart.md, 03-migration-idempotency.md. Owner: PA. Target: 2026-09-05. Status: Not Started.

Short-term (2026-Q3)

- ❑ **Start at least one new PDCA cycle** in 2026-Q3 (suggested: “deployment frequency metric” — cross-link 11). Owner: VN. Target: 2026-09-30. Status: Not Started.
- ❑ **Add a PDCA link to post-mortem template** so every post-mortem asks “should this become a cycle?”. Owner: VN. Target: 2026-09-30. Status: Not Started.
- ❑ **Quarterly cadence enforcement**: heartbeat alert that pings Venkat if no cycle started in the past 90 days. Owner: PA. Target: 2026-09-30. Status: Not Started.
- ❑ **Wire LEARNINGS.md entries to cite cycle IDs** (or vice versa). Owner: PA. Target: 2026-09-30. Status: Not Started.

Medium-term (2026-Q4)

- ❑ **First annual cycle summary** at docs/phase6/06-process-improvements/2026-summary.md. Owner: VN. Target: 2027-01-15. Status: Not Started.
- ❑ **Mature the metric linkage** — every cycle’s Check step pulls from a metric defined in 11. Owner: PA. Target: 2026-12-31. Status: Not Started.
- ❑ **At least 4 cycles completed** in 2026-Q4. Owner: PA. Target: 2026-12-31. Status: Not Started.
- ❑ **Cross-link with 12** — every cycle starts with a problem found via RCA or via post-mortem. Owner: VN. Target: 2026-12-31. Status: Not Started.

Long-term (2027+)

- ❑ **Quarterly cycle review** integrated into the quarterly review in 09. Owner: VN. Target: 2027-03-31. Status: Not Started.
- ❑ **Cycle-pattern dashboard** — count cycles by domain (backup, deploy, security, etc.). Owner: VN. Target: TBD. Status: Not Started.
- ❑ **External PDCA review** — annual external reviewer validates cycle quality. Owner: VN. Target: TBD. Status: Not Started.

Recurring verification (runs forever)

- ❑ **Quarterly cycle count check** (≥ 1 started in the past 90 days). Owner: PA. Frequency: quarterly. Status: Not Started.
- ❑ **Annual cycle summary** (1-page retrospective). Owner: VN. Frequency: annually. Status: Not Started.
- ❑ **Cycle-to-runbook-citation check** (every cycle’s Act step updates at least one runbook footer or LEARNINGS entry). Owner: PA. Frequency: per cycle. Status: Not Started.

Hypothesise, measure, standardise. If the hypothesis fails, document the failure — that’s the most valuable cycle outcome. Document or repeat.